

Marginalizing shocks

Cover that shocks which I ought not to see.

Model reparameterization

We marginalize the shocks in the mixture model! Going from:

```
# Non-shocky
target += normal_lpdf(y[n] | mu[n] + delta_sck[n], sigma)
target += normal_lpdf(delta_sck[n] | 0, inner_tau_sck)

# Shocky
target += normal_lpdf(y[n] | mu[n] + delta_sck[n], sigma)
target += normal_lpdf(delta_sck[n] | 0, outer_tau_sck)
```

to a log-likelihood where shocks are marginalized out:

```
# Non-shocky
target += normal_lpdf(y[n] | mu[n], eta)

# Shocky
target += normal_lpdf(y[n] | mu[n], eta * sqrt{1 + nu / eta^2})
```

where:

$$\eta = \sqrt{\sigma^2 + \tau_{\text{inner}}^{\text{shock}^2}}$$
$$\nu = \tau_{\text{outer}}^{\text{shock}^2} - \tau_{\text{inner}}^{\text{shock}^2}$$

In Stan, we now have in the model block:

```
writeLines(
  readLines(file.path(wd, 'model', 'stan',
    'model9_marginalshocks.stan'))[230:315])
```

```
model {
  alpha ~ normal(0, 0.69); // 0.2 mm <- exp(alpha) * 1 mm <- 5 mm
  beta_gdd ~ normal(0, log(1.8) / 2.57); // -log(1.8) <- beta_gsl <- log(1.8)
  beta_sm ~ normal(0, log(1.8) / 2.57); // -log(1.8) <- beta_gsl <- log(1.8)
  beta_vpd ~ normal(0, log(1.8) / 2.57); // -log(1.8) <- beta_gsl <- log(1.8)
  rho_sp ~ lognormal(2.65, 0.135); // 10 <- rho <- 20
  gamma_sp ~ normal(0, log(10) / 2.57); // 0 <- gamma <- log(10)
  for (s in 1:N_stands)
    f_tilde_sh[s] ~ normal(0, 1);
  rho_sh ~ lognormal(1.7, 0.26); // 3 <- rho_sh <- 10
  gamma_sh ~ normal(0, log(3) / 2.57); // 0 <- gamma_sh <- log(3)
  eta ~ gamma(6, 43.5);
```

```

nu ~ gamma(0.5, 0.033);
phi_sck ~ beta(2, 20);
omega_conc_sck ~ beta(230, 14); // 0.9 <- omega_conc_sck <- 0.97 (most trees, but not ALL trees)
omega_nonconc_sck ~ beta(1, 20); // 0 <- omega_conc_sck <- 0.15 (should be rare, but... who knows?)

vector[N] mu;
for (t in 1:N_trees) {
  array[N_years[t]] int tree_idx
  = linspace_int_array(N_years[t],
    tree_start_idx[t],
    tree_end_idx[t]);

  array[N_years[t]] int all_years_idx_tree = all_years_idx[tree_idx];
  array[N_years[t]] real years_tree = years[tree_idx];

  int stand_idx = stand_idx[t];
  int species_idx = species_idx[t];

  vector[N_years[t]] gdd_obs_tree = gdd_obs[tree_idx];
  vector[N_years[t]] sm_obs_tree = sm_obs[tree_idx];
  vector[N_years[t]] vpd_obs_tree = vpd_obs[tree_idx];

  mu[tree_idx] = alpha
  + beta_gdd[species_idx] * (gdd_obs_tree - gdd0)
  + beta_sm[species_idx] * (sm_obs_tree - sm0)
  + beta_vpd[species_idx] * (vpd_obs_tree - vpd0)
  + f_sh[stand_idx, all_years_idx_tree];

  f_tilde[tree_idx] ~ normal(0,1);
}

// Observational model with marginalized shocks
for (s in 1:N_stands) {

  array[N_stand_trees[s]] int stand_trees_idx = linspace_int_array(N_stand_trees[s],
    stand_trees_start_idx[s], stand_trees_end_idx[s]);
  vector[N_stand_years[s]] log_p0 = rep_vector(0, N_stand_years[s]);
  vector[N_stand_years[s]] log_p1 = rep_vector(0, N_stand_years[s]);

  for(ts in 1:N_stand_trees[s]){
    int t = stand_trees_idx[ts];
    array[N_years[t]] int tree_idx = linspace_int_array(N_years[t], tree_start_idx[t], tree_end_idx[t]);
    array[N_years[t]] int all_years_idx_tree = all_years_idx[tree_idx];

    for(y in 1:N_years[t]) {
      int ys = all_years_idx_tree[y]-stand_start_years_idx[s]+1;

      log_p0[ys] += log_mix(omega_nonconc_sck,
        normal_lpdf(log_rw_obs[tree_idx[y]] | mu[tree_idx[y]]
          + f[tree_idx[y], eta * sqrt(1 + nu / eta^2)),
        normal_lpdf(log_rw_obs[tree_idx[y]] | mu[tree_idx[y]]
          + f[tree_idx[y]], eta));

      log_p1[ys] += log_mix(omega_conc_sck,
        normal_lpdf(log_rw_obs[tree_idx[y]] | mu[tree_idx[y]]
          + f[tree_idx[y], eta * sqrt(1 + nu / eta^2)),
        normal_lpdf(log_rw_obs[tree_idx[y]] | mu[tree_idx[y]]
          + f[tree_idx[y]], eta));

    }
  }

  for(y in 1:N_stand_years[s]) {
    target += log_mix(phi_sck[s], log_p1[y], log_p0[y]);
  }
}
}

```

But... my shocks...

Our large shocks are likely tied to interesting ecological stuff... But by marginalizing them, we don't have inferences on them anymore.

It could be doable to “to marginalize out η in the “small shock” branch of the mixture model but leave η explicit in the “large shock” branch of the mixture model” (Mike).

Since I'm not totally sure how to do that, I decided to sample the δ_{shock} afterwards, to recover their posterior distributions in the generated quantities block. This is an ongoing project, so for now I recover only the shock state (z_t).

```
writeLines(
  readLines(file.path(wd, 'model', 'stan',
    'model9_marginalshocks.stan'))[351:405])
```

```
generated quantities {
  array[N] int sck_state; // latent state, zt = 0 or zt = 1
  array[N] real log_rw_pred;

  for (t in 1:N_trees) {
    array[N_years[t]] int tree_idx
    = linspace_int_array(N_years[t],
      tree_start_idx[t],
      tree_end_idx[t]);

    array[N_years[t]] int all_years_idx = all_years_idx[tree_idx];
    array[N_years[t]] real years_tree = years[tree_idx];

    int stand_idx = stand_idx[t];
    int species_idx = species_idx[t];

    vector[N_years[t]] gdd_obs_tree = gdd_obs[tree_idx];
    vector[N_years[t]] sm_obs_tree = sm_obs[tree_idx];
    vector[N_years[t]] vpd_obs_tree = vpd_obs[tree_idx];

    vector[N_years[t]] mu = alpha
    + beta_gdd[species_idx] * (gdd_obs_tree - gdd0)
    + beta_sm[species_idx] * (sm_obs_tree - sm0)
    + beta_vpd[species_idx] * (vpd_obs_tree - vpd0)
    + f_sh[stand_idx, all_years_idx_tree];

    // mixture weight for shock
    real mw_shock = phi_sck[stand_idx]*omega_conc_sck + (1-phi_sck[stand_idx])*omega_nonconc_sck;

    for(y in 1:N_years[t]){
      // mixture components
      real log_pshock = log(mw_shock) + normal_lpdf(log_rw_obs[tree_idx[y]] | mu[y] + f[tree_idx[y]], eta * sqrt(1 + nu / eta^2))
      real log_pshock_plus_pnonshock = log_mix(mw_shock,
        normal_lpdf(log_rw_obs[tree_idx[y]] | mu[y] + f[tree_idx[y]], eta * sqrt(1 + nu / eta^2)),
        normal_lpdf(log_rw_obs[tree_idx[y]] | mu[y] + f[tree_idx[y]], eta));

      // probability of shock state
      real lambda_shock = exp(log_pshock - log_pshock_plus_pnonshock);
      sck_state[tree_idx[y]] = bernoulli_rng(lambda_shock); // or something like categorical_rng(lambda_shock);?

      if(sck_state[tree_idx[y]] == 1) {
        log_rw_pred[tree_idx[y]] = normal_rng(mu[y] + f[tree_idx[y]], eta * sqrt(1 + nu / eta^2));
      } else {
        log_rw_pred[tree_idx[y]] = normal_rng(mu[y] + f[tree_idx[y]], eta);
      }
    }
  }
}
```

Test with a small dataset

Let's run this new model on a very small dataset, 21 trees and 1 site.

The good thing here is that I don't need to tune the (non-)centered parametrization anymore.

```
data <- readRDS(file = file.path(wd, 'output', 'model', 'data_15nov2025_az592.rds'))
data$N_stand_trees <- array(data$N_stand_trees, dim = 1)
data$N_stand_years <- array(data$N_stand_years, dim = 1)
data$stand_start_years_idx <- array(data$stand_start_years_idx, dim = 1)
data$stand_trees_end_idx <- array(data$stand_trees_end_idx, dim = 1)
data$stand_trees_start_idx <- array(data$stand_trees_start_idx, dim = 1)

if(FALSE){
  fit <- stan(file=file.path(wd, 'model', 'stan', 'model9_marginalshocks.stan'),
    data=data, seed=5838293, chains = 4,
    warmup=1000, iter=2024, refresh=10)
  saveRDS(fit, file.path(wd, 'model/shocks/output/marginal', 'fit.rds'))
}else{
```

```
fit <- readRDS(file.path(wd, 'model/shocks/output/marginal', 'fit.rds'))
}
```

Let's look at the diagnostics:

```
diagnostics <- util$extract_hmc_diagnostics(fit)
util$check_all_hmc_diagnostics(diagnostics)
```

All Hamiltonian Monte Carlo diagnostics are consistent with reliable Markov chain Monte Carlo.

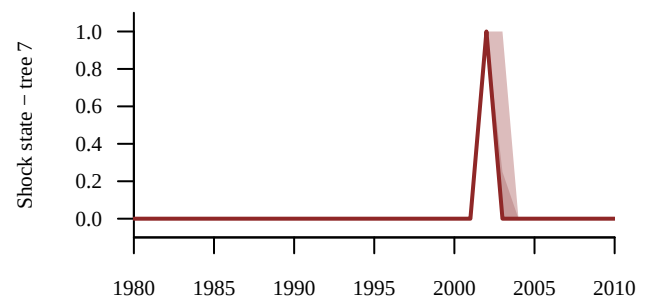
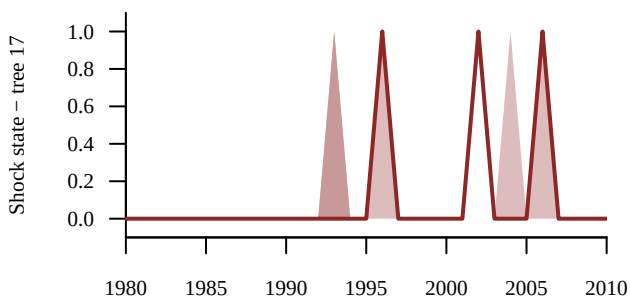
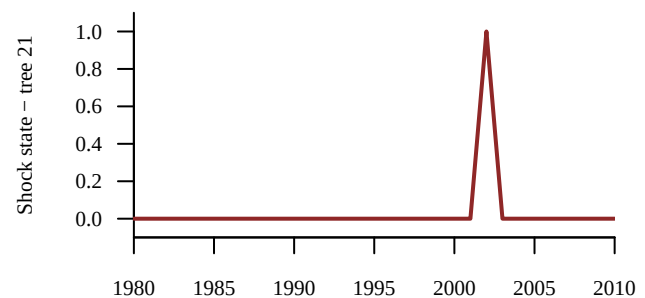
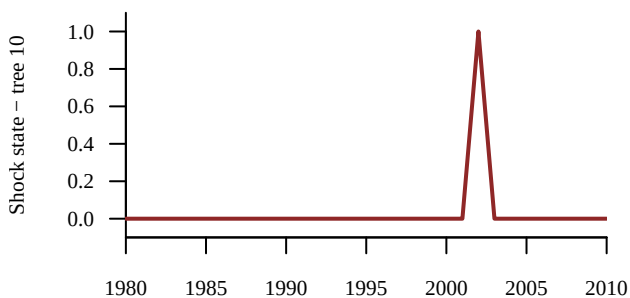
```
samples <- util$extract_expectand_vals(fit)
base_samples <- util$filter_expectands(samples,
  c('alpha',
    'beta_gdd', 'beta_sm', 'beta_vpd',
    'rho_sh', 'gamma_sh',
    'rho_sp', 'gamma_sp',
    'eta', 'nu',
    'phi_sck', 'omega_conc_sck', 'omega_nonconc_sck'),
  check_arrays = TRUE)
util$check_all_expectand_diagnostics(base_samples)
```

All expectands checked appear to be behaving well enough for reliable Markov chain Monte Carlo estimation.

Clean! Let's continue.

These are the latent shock states:

```
set.seed(123456)
random_trees <- sample(1:data$N_trees, 4)
par(mfrow = c(2,2), mar = c(2, 5, 3, 1))
for(t in random_trees){
  idxs_t <- data$tree_start_idxes[t]:data$tree_end_idxes[t]
  names <- sapply(idxs_t,
    function(x) paste0('sck_state[', x, ']'))
  util$plot_conn_pushforward_quantiles(samples, names, data$years[idxs_t],
    xlab="", ylab=paste0("Shock state - tree ",t),
    display_ylim=c(-0.1, 1.1), display_xlim = c(1980, 2010))
}
```

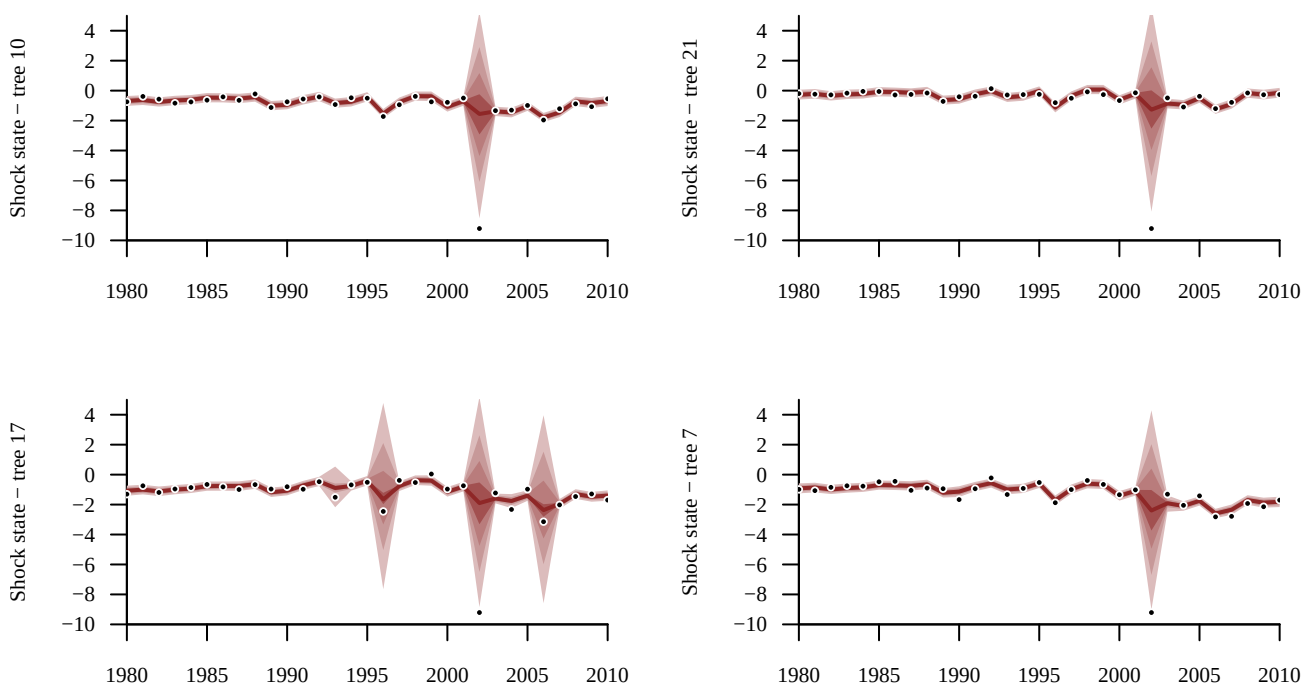


Next step

Is it possible to recover the posterior of $\delta_{\text{shock}}(k) \mid y(k)$, without having $\tau_{\text{inner}}^{\text{shock}}$ or $\tau_{\text{outer}}^{\text{shock}}$?
 Maybe a first step could be to fix: $\tau_{\text{inner}}^{\text{shock}} = 0$?

Because for now my predictions look like:

```
par(mfrow = c(2,2), mar = c(2, 5, 3, 1))
for(t in random_trees){
  idxs_t <- data$tree_start_idx[t]:data$tree_end_idx[t]
  names <- sapply(idxs_t,
    function(x) paste0('log_rw_pred[', x, ']'))
  util$plot_conn_pushforward_quantiles(samples, names, data$years[idxs_t],
    xlab="", ylab=paste0("Shock state - tree ",t),
    display_ylim=c(-10, 5), display_xlim = c(1980, 2010))
  points(data$years[idxs_t], data$log_rw_obs[idxs_t], pch=16, cex=0.7, col="white")
  points(data$years[idxs_t], data$log_rw_obs[idxs_t], pch=16, cex=0.4, col="black")
}
```



Assuming that $\tau_{\text{inner}} = 0$

If I assume that $\tau_{\text{inner}}^{\text{shock}} = 0$, I can reconstruct the large shocks using the normal-normal conjugacy (*merci* Google):

$$\delta_{\text{shock}}(k) \mid y_k \sim \text{normal} \left(\left(\tau_{\text{outer}}^{\text{shock}^2} / (\tau_{\text{outer}}^{\text{shock}^2} + \sigma^2) \right) \times y_k, \sqrt{(\tau_{\text{outer}}^{\text{shock}^2} \times \sigma^2) / (\tau_{\text{outer}}^{\text{shock}^2} + \sigma^2)} \right)$$

In the generated quantities block, we can then have:

```
writeLines(
  readLines(file.path(wd, 'model', 'stan',
    'model9_marginalshocks_zerotauninner.stan'))[395:405])
```

```

if(sck_state[tree_idx[y]] == 1) {
  // we can reconstruct shock posterior using the normal-normal conjugacy
  real conjugate_mean = (outer_tau_sck^2 / (outer_tau_sck^2 + sigma^2)) * log_rw_obs[tree_idx[y]];
  real conjugate_sd = sqrt((outer_tau_sck^2 * sigma^2) / (outer_tau_sck^2 + sigma^2));
  delta_sck[tree_idx[y]] = normal_rng(conjugate_mean, conjugate_sd);
  log_rw_pred[tree_idx[y]] = normal_rng(mu[y] + f[tree_idx[y]] + delta_sck[tree_idx[y]], sigma);
}else{
  log_rw_pred[tree_idx[y]] = normal_rng(mu[y] + f[tree_idx[y]], sigma);
}

```

```

if(FALSE){
  fit2 <- stan(file=file.path(wd, 'model', 'stan', 'model9_marginalshocks_zerotauninner.stan'),
    data=data, seed=5838293, chains = 4,
    warmup=1000, iter=2024, refresh=10)
  saveRDS(fit2, file.path(wd, 'model/shocks/output/marginal', 'fit2.rds'))
}else{
  fit2 <- readRDS(file.path(wd, 'model/shocks/output/marginal', 'fit2.rds'))
}

```

Diagnostics are cleaned:

```

diagnostics <- util$extract_hmc_diagnostics(fit2)
util$check_all_hmc_diagnostics(diagnostics)

```

All Hamiltonian Monte Carlo diagnostics are consistent with reliable Markov chain Monte Carlo.

```

samples <- util$extract_expectand_vals(fit2)
base_samples <- util$filter_expectands(samples,
  c('alpha',
    'beta_gdd', 'beta_sm', 'beta_vpd',
    'rho_sh', 'gamma_sh',
    'rho_sp', 'gamma_sp',
    'sigma', 'outer_tau_sck',
    'phi_sck', 'omega_conc_sck', 'omega_nonconc_sck'),
  check_arrays = TRUE)
util$check_all_expectand_diagnostics(base_samples)

```

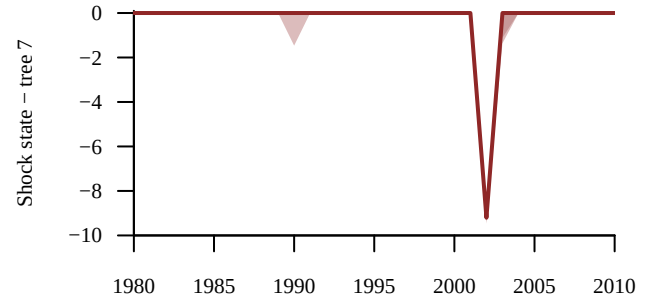
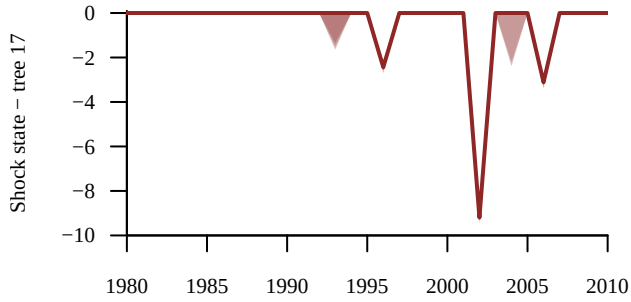
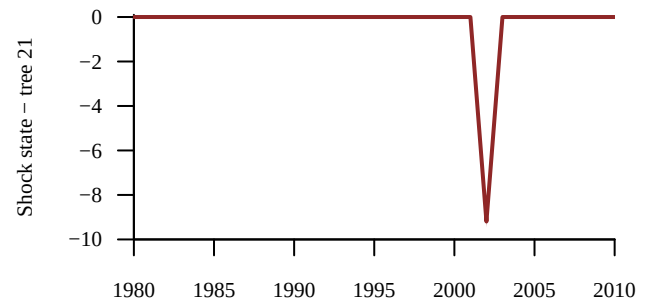
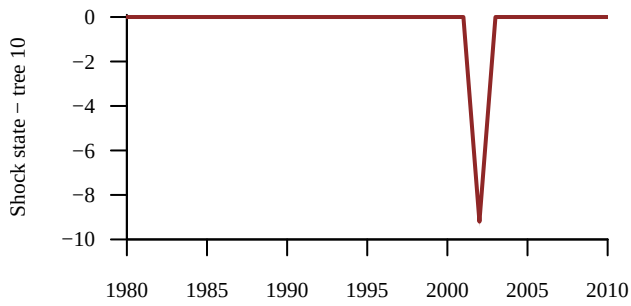
All expectands checked appear to be behaving well enough for reliable Markov chain Monte Carlo estimation.

We can look at the latent shocks:

```

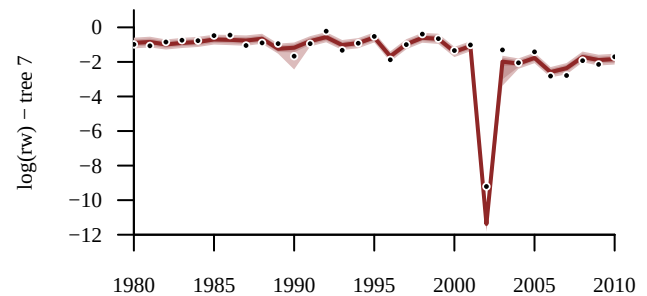
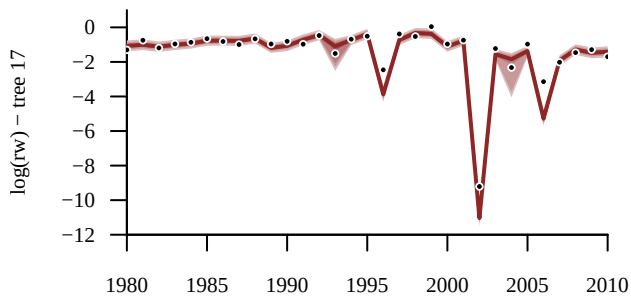
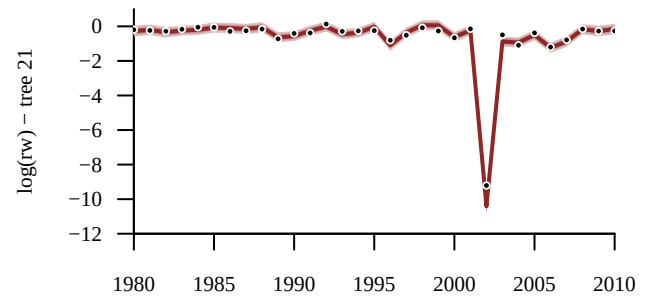
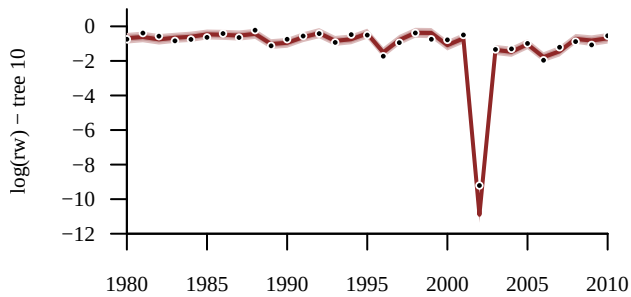
par(mfrow = c(2,2), mar = c(2, 5, 3, 1))
for(t in random_trees){
  idxs_t <- data$tree_start_idx[t]:data$tree_end_idx[t]
  names <- sapply(idxs_t,
    function(x) paste0('delta_sck[', x, ']'))
  util$plot_conn_pushforward_quantiles(samples, names, data$years[idxs_t],
    xlab="", ylab=paste0("Shock state - tree ",t),
    display_ylim=c(-10, 0.1), display_xlim = c(1980, 2010))
}

```



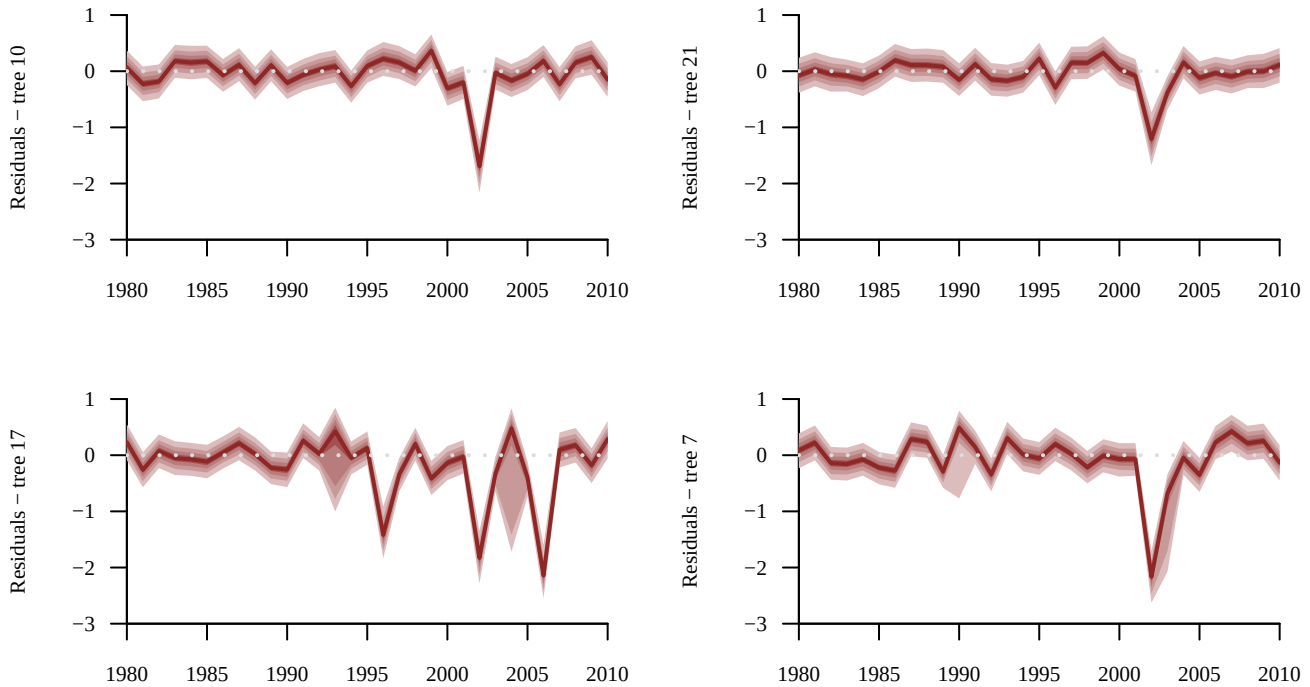
And at the predictions:

```
par(mfrow = c(2,2), mar = c(2, 5, 3, 1))
for(t in random_trees){
  idxs_t <- data$tree_start_idx[t]:data$tree_end_idx[t]
  names <- sapply(idxs_t,
    function(x) paste0('log_rw_pred[', x, ']'))
  util$plot_conn_pushforward_quantiles(samples, names, data$years[idxs_t],
    xlab="", ylab=paste0("log(rw) - tree ",t),
    display_ylim=c(-12, 1), display_xlim = c(1980, 2010))
  points(data$years[idxs_t], data$log_rw_obs[idxs_t], pch=16, cex=0.7, col="white")
  points(data$years[idxs_t], data$log_rw_obs[idxs_t], pch=16, cex=0.4, col="black")
}
```



However, it seems that the shocks are bit too large...

```
par(mfrow = c(2,2), mar = c(2, 5, 3, 1))
for(t in random_trees){
  idxs_t <- data$tree_start_idx[t]:data$tree_end_idx[t]
  names <- apply(idxs_t,
                 function(x) paste0('log_rw_pred[', x, ']'))
  util$plot_conn_pushforward_quantiles(samples, names, data$years[idxs_t],
                                     baseline_values = data$log_rw_obs[idxs_t],
                                     xlab="", ylab=paste0("Residuals - tree ",t),
                                     display_ylim=c(-3, 1), display_xlim = c(1980, 2010),
                                     residual = TRUE)
}
```



And, let's not forget the stand-level GP, which is still wiggly:

```
s <- 1
names <- apply(1:data$N_all_years,
               function(y) paste0('f_sh[', s, ',', y, ']'))
util$plot_realizations(samples, names, plot_xs = data$all_years[1:data$N_all_years], N_plots = 50,
                      main = paste0('Stand ', data$uniq_stand_ids[s]), ylab = 'f_sh',
                      display_xlim = data$all_years[c(1,data$N_all_years)], display_ylim = c(-2,2))
```

